Global Warming Science

https://courses.seas.harvard.edu/climate/eli/Courses/EPS101/

Heat waves

Please use the template below to answer the workshop questions. "XX" indicates places where you need to complete/write code or add a discussion.

Your name:

[ ]:
```
# Load libraries and data:
import numpy as np
import pickle
import matplotlib.pyplot as plt
from scipy import stats
from scipy.stats import linregress
from scipy.signal import savgol_filter # for smoothing time series
import warnings
import cartopy.crs as ccrs
import datetime
from datetime import timedelta
import matplotlib.dates as mdates
from scipy.signal import savgol_filter # for smoothing time series
from scipy import optimize
from matplotlib.ticker import (MultipleLocator, FormatStrFormatter,
                               AutoMinorLocator)

# load the data from a pickle file:
with open('./heat_waves_variables.pickle', 'rb') as file:
    while 1:
        try:
            d = pickle.load(file)
        except (EOFError):
            break
            # print information about each extracted variable:
        for key in list(d.keys()):
            print("extracting pickled variable: name=", key,  "; size=", d[key].shape)
            #print("type=",type(d[key]))
        globals().update(d)
```

Explanation of input variables:

Siberia heat wave observations:

Verhojansk_Tmax: Siberian daily-maximum temperature records

Verhojansk_Tmax_dates: can be used to plot Verhojansk_Tmax vs time.

Verhojansk_Tmax_day_of_year: showing day of year from 1 to 366.

Model Tmax time series for multiple ensemble members:

heat_waves_Tmax_all_timeseries: time series of maximum daily temperatures over the Great Plains region of the US from 1920 to 2100 for 31 ensemble members of historical simulation followed by RCP8.5, from the CESM climate model.

heat_waves_ensemble_members_dates: dates for time series in heat_waves_Tmax_all_timeseries

Data as a function of time and longitude/latitude for calculating composites over region of heat waves:

heat_waves_weekly_averages_TREFHTMX, heat_waves_weekly_averages_Z500: weekly averaged data of daily maximum temp and height of the 500 mb surface,

as a function of latitude and longitude, for all augusts from 1920 to 2100. The structure of these data is explained below in the notebook.

heat_waves_lat, heat_waves_lon: lat/lon for calculating and contouring composites

1) The Siberian heat wave of summer 2020.

(a) Plot the daily maximum temperature climatology for Verhojansk, Siberia from January to December and then superimpose the 2020 temperature.

(b) Plot the anomaly time series, calculated as the deviation from the daily climatology, and use it to calculate the peak time, amplitude and length of the summer heat wave based on what you think are reasonable thresholds.

[ ]:
```
# calculate daily climatology:

Verhojansk_Tmax_climatology=np.zeros(365)
for day in np.arange(1,366):
    Verhojansk_Tmax_climatology[day-1]=np.mean(Verhojansk_Tmax[XX])

# smooth the climatology: # arguments are window_size,polynomial_order
Verhojansk_Tmax_climatology_nonsmooth = Verhojansk_Tmax_climatology
Verhojansk_Tmax_climatology = \
  savgol_filter(Verhojansk_Tmax_climatology, 21, 2)

Verhojansk_Tmax_anomaly=Verhojansk_Tmax*np.nan
Verhojansk_Tmax_climatology_all_years=Verhojansk_Tmax*np.nan
for day in range(len(Verhojansk_Tmax)):
    iday=Verhojansk_Tmax_day_of_year[day]-1
    Verhojansk_Tmax_anomaly[day]=XX
    Verhojansk_Tmax_climatology_all_years[day]=Verhojansk_Tmax_climatology[iday]

# plot daily climatology and anomaly timeseries:
plt.figure(figsize=(6,6),dpi=600)

plt.subplot(2,1,1)
# plot climatology over one year
plt.plot(XX)
plt.xlabel("XX")
plt.ylabel("XX")
plt.title("XX")


plt.subplot(2,1,2)
# plotting the anomaly:
T1=  # Tmax climatology
T2=  # Tmax itself
plt.fill_between(XX,XX,XX,where=XX>=XX \
```

```
                ,interpolate=True,lw=0.3,color="r")
plt.fill_between(XX,XX,XX,where=XX>=XX \
                ,interpolate=True,lw=0.3,color="b")
plt.xlabel("XX")
plt.ylabel("XX")
plt.title("XX")
plt.xlim(datetime.datetime.strptime("2020-01-01", '%Y-%m-%d')\
        ,datetime.datetime.strptime("2020-09-01", '%Y-%m-%d'))
# can make x-axis labeling nicer with these:
#plt.gca().xaxis.set_major_formatter(mdates.DateFormatter('%m/%d/%Y'))
#plt.xticks(rotation=XX,fontsize=XX,ha=XX)
```

[ ]:
```
# plot the anomaly plus climatology a-la Ciavarella-etal-2020, world weather␣
 ↪attribution project:
# lower panel of Fig 1, in pdf copy available at
# https://link.springer.com/article/10.1007/s10584-021-03052-w,
# see also https://www.worldweatherattribution.org/
 ↪siberian-heatwave-of-2020-almost-impossible-without-climate-change
fig=plt.figure(figsize=(3.6,2.2),dpi=600)
plt.subplot(3,1,3)
# plot anomaly only:
plt.plot(XX,XX)
plt.xlabel("XX")
plt.ylabel("XX")
plt.title("XX")
plt.xlim(XX,XX)
plt.ylim(XX,XX)
plt.xticks(XX)

plt.tight_layout()
```

2 Heat stress and the wet bulb temperature.

(a) Calculate the wet bulb temperature (to within 0.001 °C) for an air parcel with temperature of 37 °C and relative humidity of 60%: evaluate the appropriate energy balance for different values of T_w until you find the one that satisfies it.

[ ]:
```
c_p  = 1005     # J/kg/K
L       = 24.93e5  # latent heat J/kg (vaporization at t.p.)
P=1000

def q_sat(T,P):
    """
    Calculate saturation specific humidity (gr water vapor per gram moist air).
    inputs:
    T: temperature, in Kelvin
    P: pressure, in mb
    """
    qw=XX
    return qw

T=37  # Celsius, need to convert to Kelvin to be used in the function q_sat
rh=0.60
```

```
# try different values of T_w until the RHS equals the LHS:
T_w=XX # Celsius, need to convert to Kelvin to be used in the function q_sat
RHS=XX
LHS=XX
print("RHS=",RHS,", LHS=",LHS,", RHS-LHS=",RHS-LHS)
```

(b) Optional extra credit: Calculate and contour the WBT as a function of relative humidity and temperature.

[ ]:
```
# define a function that calculates WBT from the above heat budget

cp_dry  = 1005     # J/kg/K
c_p=1.0*cp_dry
L       = 24.93e5  # latent heat J/kg (vaporization at t.p.)
P=1000

def cost(T_w,rh,T):
    # rh - percent
    # T_w, T: Celsius
    J=(c_p*T+L*(rh/100)*q_sat(273.15+T,P))  -  (XX))
    return J


def my_WBT(rh,T):
    # input: scalars; rh: percent, T: Celsius
    sol = optimize.root(cost, T, args=(rh,T))
    T_w = sol.x[0]
    return T_w


# calculate WBT(rh,T):
# --------------------
rh=np.arange(0,100.1,0.1)
T=np.arange(30,50.1,0.1)
x,y=np.meshgrid(T,rh)
my_WBT_plot=np.zeros(x.shape)

irh=-1
for rh1 in rh:
    irh=irh+1
    iT=-1
    for T1 in T:
        iT=iT+1
        my_WBT_plot[irh,iT]=my_WBT(rh1,T1)

print("done.")
```

[ ]:
```
# Contour WBT as function of RH and T:
# ------------------------------------
fig=plt.figure(figsize=(4,3),dpi=300)
levels=np.arange(15,35.1,0.1)
plt.contour(x,y,my_WBT_plot,levels=levels)
hcb=plt.colorbar(label="°C")
plt.xlabel("Temperature (°C)")
```

```
plt.ylabel("Relative humidity (%)");
```

3. Understanding heat wave statistics for an RCP8.5 projection:

(a) Plot simulated temperature time series for all ensemble members for 5 yr periods starting in 1920, 2010, and 2090.

[ ]:
```
# note that heat_waves_Tmax_all_timeseries is in K, convert to C as needed:
Tmax_all_timeseries=heat_waves_Tmax_all_timeseries

years=heat_waves_ensemble_members_dates
ensemble_members=range(0,len(heat_waves_Tmax_all_timeseries[:,0]))
# calc avereage over all ensemble members:
Tmax_timeseries_avg=np.mean(Tmax_all_timeseries[:,:],axis=0)

# plot Tmax timeseries
fig=plt.figure(dpi=300,figsize=(4,6))
plt.clf()

plt.subplot(3,1,1)
iensemble_member=-1
for ensemble_member in ensemble_members:
    iensemble_member=iensemble_member+1
    plt.plot(XX,XX,lw=0.25,alpha=0.4)
plt.plot(XX,XX,lw=0.4,color='b',label="mean")
plt.ylabel(XX)
plt.xlim(datetime.date(year=1920,month=1,day=1),datetime.
  ↪date(year=1925,month=1,day=1));
plt.ylim([10,47]);
plt.legend()
plt.grid(lw=0.25);

plt.subplot(3,1,2) # 2000-2010
iensemble_member=-1
for XX:

plt.plot(XX,XX,lw=0.4,color='b',label="mean")
plt.ylabel(XX)
plt.xlim(XX,XX);
plt.ylim([10,47]);
plt.legend()
plt.grid(lw=0.25);

plt.subplot(3,1,3)
XX


plt.tight_layout()
plt.show();
```

(b) Calculate and plot PDFs of maximum daily surface temperatures for 1920-1950 and then for 2070-2100. Superimpose on the 2070-2100 plot a line for the PDF calculated from the time series of 1920-1950 plus a constant temperature increase corresponding to the mean difference between the two periods. [Hint: This is recreating Figure 13.7: "Understanding changes in heat wave

statistics" from the course notes.]

(c) Calculate and plot the CDFs for the maximum daily temperature for 1920-1950 (blue bars) and then for 2070-2100 (red bars). Superimpose a line for the CDF calculated from the time series of 1920-1950 plus a constant temperature increase corresponding to the mean difference between the two periods. Use the CDFs to calculate the expected probability of exceeding maximum daily temperature in the latter period, whose probabilities were 0.1% in the earlier period.

[ ]:
```
# for both 3b and 3c:

dates=heat_waves_ensemble_members_dates
years=np.asarray([date.year for date in dates])

# limit to desired year range:
Tmax_for_pdf=Tmax_all_timeseries[:,years<=1950]-273.15
# reshape temperature data into a single array:
Tmax_for_pdf.reshape(np.prod(Tmax_for_pdf.shape))

# calculate the histograms:
Tmax_hist, Tmax_bin_edges = np.histogram(Tmax_for_pdf, bins=range(XX), density=True)
# center of bins, used for x-axis when plotting and calculating CDF:
Tmax_x=(Tmax_bin_edges[1:]+Tmax_bin_edges[0:-1])/2
# width of bins, used for calculating PDF:
Tmax_dx=(Tmax_bin_edges[1:]-Tmax_bin_edges[0:-1])
# CDF=cumulative sum over Tmax_hist*dx:
Tmax_cdf=np.cumsum(XX)

# print the temperature of an event whose probability is 0.1%
print("temperatures of events whose probability is less than 0.1%:")
print(Tmax_x[100-100*Tmax_cdf<=0.1])

# Tmax pdfs:
fig=plt.figure(figsize=(7,3),dpi=300)
plt.subplot(1,2,1)
h1=plt.bar(Tmax_x,Tmax_hist,color="b");
plt.xlabel("$T_{\\rm max}$ ($^\circ$C)")
plt.ylabel("PDF")
plt.grid(lw=0.25);

plt.subplot(1,2,2)
h1=plt.bar(Tmax_x,Tmax_cdf,color="b");
plt.xlabel("$T_{\\rm max}$ ($^\circ$C)")
plt.ylabel("CDF")
plt.grid(lw=0.25);

plt.tight_layout()
```

[ ]:
```
# now repeat for year>=2070:
# limit to desired year range:
XX

# calculate the histograms:
XX
```

```
# print the temperature of an event whose probability is 0.1%
print("probability of events whose temeprature is above that that was 0.1% before␣
 ↪warming:")
print(np.max(100-100*Tmax_cdf[XX]))

# now repeat the calculation of the histogram for year<1950, this time adding the mean
# temperature difference between the two periods:
# First, find mean temperature difference between 2070-2100 and 1920-1950:
warming=np.mean(XX)-np.mean(XX)
print("mean warming between two periods is",warming,"degree C.")
# limit to desired year range:
Tmax_for_pdf_shifted=Tmax_all_timeseries[:,years<1950]-273.15+warming
# reshape temperature data into a single array:
Tmax_for_pdf_shifted.reshape(np.prod(Tmax_for_pdf_shifted.shape))

# calculate the histograms:
Tmax_hist_shifted, Tmax_bin_edges_shifted = XX
Tmax_x_shifted=XX
Tmax_dx_shifted=XX
Tmax_cdf_shifted=np.cumsum(XX)

# plot:
# Tmax pdfs:
fig=plt.figure(figsize=(7,3),dpi=300)
plt.subplot(1,2,1)
h1=plt.bar(Tmax_x,Tmax_hist,color="b");
plt.xlabel(XX")
plt.ylabel("PDF")
plt.plot(XX,XX,color="g",drawstyle='steps');

# Tmax CDFs:
plt.subplot(1,2,2)
XX
```

(d) The two complementary views of extreme event statistics: (i) Calculate the return time of a 43 °C daily maximum temperature event for 1920–1950 and then for 2070–2100. (ii) Calculate the daily maximum temperature of an event with a return time of 10 years for 1920–1950. Can you calculate that for 2070–2100? Why? Answer:

The relevant variables are Tmax_cdf_1920 and Tmax_cdf_1970. The CDF can be used to calculate the return time τ_{return} as follows: In this case the data are daily maximum temperature, given every day. Therefore, the number of events per year that exceed T, and therefore the number of years between such events are,

$$N_{\text{days in year}} = (1 - CDF(T)) \times 365$$
$$\tau_{\text{return}}(T) = 1/N_{\text{days in year}}$$

[ ]:
```
# calculate number of days per year of T_max events:
N_days_in_year_1920=
N_days_in_year_2070=
```

```
# calculate return times:
tau_return_1920=XX
tau_return_2070=XX

# plot:
fig=plt.figure(figsize=(5,3),dpi=300)
plt.ylabel("XX")
plt.xlabel("XX");
plt.bar(Tmax_x,tau_return_1920,color="b");
plt.bar(XX,color="r",alpha=0.5);
plt.gca().set_yscale('log')
plt.xlim(25,48);
```

(e) Optional extra credit: calculate and plot the PDFs of daily maximum temperature, heat wave duration and heat wave amplitudes following the above guidelines in item (b).

[2]:
```
# XX [a fairly complex programming challenge]
```

4. Heat wave composite analysis in the region of your city of birth:

(a) Calculate and plot a climatology of August daily maximum temperature, and of Z500 (height of the 500 hPa surface).

[ ]:
```
# determine region of world to zoom in on (use an area of about 50 by 50 degree)
long_min = XX
long_max = XX
lat_min = XX
lat_max = XX

# printout to see what data look like:
print("date of week 3 data:",heat_waves_weekly_averages_TREFHTMX[3][0])
print("size of week 3 data:",heat_waves_weekly_averages_TREFHTMX[3][1].shape)
N=len(heat_waves_weekly_averages_TREFHTMX) # number of weeks in data set
nx,ny=heat_waves_weekly_averages_TREFHTMX[0][1].shape # size of data for one week
lon=heat_waves_lon
lat=heat_waves_lat

# calculate averages over all weeks (climatology):
TREFHTMX_climatology=np.zeros((nx,ny))
Z500_climatology=np.zeros((nx,ny))
for iweek in range(N):
    TREFHTMX_climatology=XX
    Z500_climatology=XX


# plot climatologies:
# -------------------
projection=ccrs.PlateCarree(central_longitude=0.0);
fig,axes=plt.subplots(1,2,figsize=(7,3),dpi=300,subplot_kw={'projection': projection});

axes[0].set_extent([long_min, long_max, lat_min, lat_max], crs=ccrs.PlateCarree())
axes[0].coastlines(resolution='110m',lw=0.3)
plt.set_cmap('bwr')
DATA=1.0*TREFHTMX_climatology
levels=np.arange(275,320,1)
```

```
c=axes[0].contourf(lon,lat, DATA[:,:],levels=levels)
clb=plt.colorbar(c, shrink=0.5, pad=0.02,ax=axes[0])
clb.set_label('K')
axes[0].set_title('$T_{max}}$ climatology');

axes[1].set_extent([long_min, long_max, lat_min, lat_max], crs=ccrs.PlateCarree())
axes[1].coastlines(resolution='110m',lw=0.3)
plt.set_cmap('bwr')
DATA=1.0*Z500_climatology/1000
levels=np.arange(5.4,6,0.01)
c=axes[1].contourf(lon,lat, DATA[:,:],levels=levels)
clb=plt.colorbar(c, shrink=0.5, pad=0.02,ax=axes[1])
clb.set_label('km')
axes[1].set_title('$Z_{500}}$ climatology');
```

(b) Calculate a time series over the city of interest (averaging over an area of about 5×5 degree).

(c) Use the time series to calculate and plot a composite of both fields plotted in (a) for heat wave events. Define a heat wave as having temperature of more than an appropriate threshold (ignoring the duration condition for simplicity).

[ ]:
```
# first, time series of weekly-averaged daily max August temperatures
# at location of interest, choose an area that is some 10x10 degree large:
Tmax_weekly_timeseries=[]
lat_range=np.logical_and(lat>=XX,lat<=XX)   # latitude range, for northeast: 40,45
lon_range=np.logical_and(lon>=360-XX,lon<=360-XX) # longitude (0-360) range, for␣
 ↪northeast: 360-75,360-70
locations = np.ix_(lat_range,lon_range)
for iweek in range(N):
    Tmax_weekly_timeseries.append(np.mean(
        heat_waves_weekly_averages_TREFHTMX[iweek][1][locations]))

mydates=np.asarray([date[0] for date in heat_waves_weekly_averages_TREFHTMX])
# convert from a list to a numpy array:
Tmax_weekly_timeseries=np.asarray(Tmax_weekly_timeseries)

# choose an appropriate threshold (experiment, based on plot below)
threshold=300.5

# plot time series of daily max temperature over region of interest:
# ------------------------------------------------------------------
plt.figure(figsize=(5,3),dpi=300)
plt.plot(XX plot the time series XX,".",markersize=3,color='b',lw=0.5,label="TREFHTMX")
plt.plot(XX plot a line at the threshold XX,"-",color='r',lw=0.5,label="threshold")
plt.title("daily max temperature over region of interest")
plt.xlabel("year")
plt.ylabel("TREFHTMX (K)")
plt.legend()
plt.grid(lw=0.25)


# next, calculate a mask time series, with 1 for days with temperature over threshold
# during 1920-1980, 0 otherwise:
```

```
mask=np.zeros(Tmax_weekly_timeseries.shape,dtype=np.int)
mask[XX set threshold condition XX]=1
```

[ ]:
```
# calculate composite:
# --------------------
TREFHTMX_composite=np.zeros((nx,ny))
Z500_composite=np.zeros((nx,ny))
# number of hot days to average over:
Navg=np.nansum(mask)
for iweek in range(N):
    if mask[iweek]==1:
        TREFHTMX_composite=XX
        Z500_composite=XX


# plot composites:
# ----------------
projection=ccrs.PlateCarree(central_longitude=0.0);
fig,axes=plt.subplots(1,2,figsize=(7,3),dpi=300,subplot_kw={'projection': projection});

XX define axes for first plot etc
DATA=1.0*(TREFHTMX_composite-TREFHTMX_climatology)
XX contour

XX second plot
DATA=1.0*(Z500_composite-Z500_climatology)
XX contour
```

5. Decadal heat wave statistics.

(a) Calculate and plot decadal time series of the number of heat waves days over the great plains from 1920 to 2100.

[ ]:
```
Ndecades=int(np.floor((years[-1]-years[0])/10))

# initialize arrays:
Nhot_days_per_year=np.zeros(Ndecades)
year_decade_start=np.zeros(Ndecades)

# calculate for each decade:
for idec in range(Ndecades):
    year_start=years[0]+idec*10
    year_end=XX
    year_decade_start[idec]=year_start+5 # mid of decade for plotting
    # heat wave days per year: create an array Tmax that is 1 if temperature
    # is above 40C, nan otherwise
    # first, initialize Tmax to temperature in Celisus:
    Tmax=1.0*Tmax_all_timeseries[:,np.logical_and(years<XX,years>=XX)]-273.15
    # count number of days above 40C:
    heatwave_threshold=40.0
    heatwave_counter = 0 # set up counter variable to increase whenever T threshold met
    for T in Tmax.flatten(): # flatten makes T_array 1D to iterate over with the for␣
 ↪loop easily
        if XX:
```

```
            heatwave_counter = XX # increase counter by 1 when temperature > threshold
    Nhot_days_per_year[idec]=heatwave_counter
    # normalize by number of ensemble members and number of years per decade:
    Nhot_days_per_year[idec]=Nhot_days_per_year[idec]/len(ensemble_members)/10.0

# plot number of hot days per decade
fig=plt.figure(figsize=(7,3),dpi=300)
h1=plt.plot(XX,XX,color="b");
plt.xlabel("year")
plt.ylabel("# days above 40C per year")
plt.grid(lw=0.25);
plt.tight_layout();
```

(b) ***Optional extra credit:*** **calculate and plot the decadal statistics of the averaged amplitude, duration and frequency (number of events per year) of heat waves for 1920–2100.**

[ ]:
```
# first, calculate a mask timeseries indicating which days are above the temperature␣
  ↪threshold:
Tmax_threshold=40
Ndays_threshold=3
XX
Tmax_mask_days_above_threshold=XX

# next, a mask indicating which days are part of a heat wave event taking into
# account the minimum temperature and minimum duration threshold:
iensemble_member=-1
for ensemble_member in ensemble_members:
    iensemble_member=iensemble_member+1
    XX loop over all days in each ensemble member, see if each day is part of a
    XX    continuous period of days above threhsold temperature:

# now calculate decadal heat wave statistics:
XX

# plot time series of decadal statistics:
# ---------------------------------------

fig=plt.figure(figsize=(6,4),dpi=300)
plt.subplot(2,2,1)
plt.plot(year_decade_start,Ndays_heat_waves_per_year)
plt.ylabel("# days/year")
#plt.xlabel("year");
plt.xlim(1920,2100)
plt.grid(lw=0.25)

plt.subplot(2,2,2)
plt.plot(year_decade_start,avg_Tmax_heat_waves)
plt.ylabel("$\\langle T_{\\rm max}\\rangle$, ($^\circ$C)")
#plt.xlabel("year");
plt.xlim(1920,2100)
plt.grid(lw=0.25)

plt.subplot(2,2,3)
plt.plot(year_decade_start,num_heat_waves_per_year)
```

```
plt.ylabel("# events/year")
plt.xlabel("year");
plt.xlim(1920,2100)
plt.grid(lw=0.25)

plt.subplot(2,2,4)
plt.plot(year_decade_start,avg_duration_heat_waves)
plt.ylabel("averaged duration")
plt.xlabel("year");
plt.xlim(1920,2100)
plt.grid(lw=0.25)

plt.tight_layout()
plt.show();
```

[ ]: